A Comparative Study of Value-Based and Policy-Based Reinforcement Learning on CartPole-v1

Het Jethva
Department of Computer Science
INSTITUTION NAME
COURSE CODE
hetjethva07@gmail.com

Abstract—
Index Terms—reinforcement learning, Q-learning, deep Q-network, policy gradient, function approximation, empirical comparison

I. Introduction

CartPole [1] is the standard first problem in reinforcement learning. It is small, it is fast, and every general-purpose method solves it. That last property is what makes it interesting for a comparison: because the outcome is not in doubt, the question stops being which method works and becomes what each method costs to make it work.

This paper compares three methods that sit at three levels of approximation:

- Tabular Q-learning [2], which represents the value function exactly over a discretised state space;
- DQN [3], which replaces the table with a neural network and adds the two devices—a replay buffer and a target network—that make that replacement stable;
- REINFORCE with a learned baseline [4], which abandons the value function as the object of interest and optimises the policy directly.

The comparison is not a ranking. All three methods reach a working policy on CartPole-v1, so a claim of the form "method X is best" would be both uninformative and already known. The contribution here is a trade-off map: a measurement of what each method spends to arrive at that policy, along four axes that are usually reported separately or not at all—environment samples, computation, reliability across random seeds, and implementation effort.

Two design commitments make the resulting numbers comparable. First, every method runs under a single shared training loop with an identical budget of 100k environment steps and an identical evaluation procedure, so no method can gain an advantage from the harness. Second, hyperparameters are selected by a random search with an equal trial budget per method, on seeds disjoint from those used for the reported results. Without the second commitment, a comparison paper measures how the author distributed their attention.

The contributions are:

1) A controlled comparison of the three methods on CartPole-v1 over 10 seeds each, under a shared budget and a fixed evaluation set.
2) A trade-off table quantifying sample cost, compute cost, reliability and implementation cost together, including the implementation-cost axis that empirical RL papers normally omit.
3) A sensitivity study for each method over the hyperparameter that carries its characteristic failure mode, and ablations of the two components that distinguish the deep methods from their textbook forms.
4) A complete, seed-reproducible implementation with the full result set.

II. Background

A. Problem setting

Reinforcement learning problems are modelled as a Markov decision process $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $P(s' \mid s, a)$ is the transition distribution, $r(s, a)$ the reward, and $\gamma \in [0, 1)$ a discount factor. The agent seeks a policy $\pi(a \mid s)$ maximising the expected discounted return $\mathbb{E}[\sum_t \gamma^t r_t]$ [5].

CartPole-v1 has a four-dimensional continuous observation—cart position, cart velocity, pole angle and pole angular velocity—and two actions, pushing the cart left or right. Every timestep yields a reward of 1. An episode terminates when the pole angle exceeds 12° or the cart leaves the track, and is truncated at 500 steps. The maximum achievable return is therefore 500, and the conventional definition of solving the task is a mean return of at least 475.

B. Tabular Q-learning

Q-learning [2] estimates the optimal action-value function by the off-policy update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right], \tag{1}$$

which converges to Q^* under standard conditions when every state–action pair is visited infinitely often and Q is represented exactly.

That last requirement is the difficulty here. An exact representation needs a finite state space, and CartPole's is continuous, so the observation must be discretised into bins. Two of its four dimensions—the two velocities—are unbounded, so they must additionally be clipped before they can be binned. The resulting agent therefore does not solve CartPole; it solves a discretised approximation of CartPole, and the gap between the two is a cost that the method pays and the other two do not.

C. Deep Q-networks

DQN [3] replaces the table with a parameterised function $Q_\theta(s, a)$ and minimises the temporal-difference error against a bootstrap target. Two devices stabilise what is otherwise a moving-target regression:

- a replay buffer, which stores past transitions and samples training batches uniformly from them, breaking the temporal correlation that would otherwise violate the i.i.d. assumption of stochastic gradient descent;
- a target network Q_{θ^-}, a periodically refreshed copy of the online network used to compute the bootstrap term, which stops the regression target from moving with every gradient step.

The loss on a sampled transition is

$$\mathcal{L}(\theta) = \ell_\delta \Big(Q_\theta(s, a) - \Big[r + \gamma(1 - d) \max_{a'} Q_{\theta^-}(s', a') \Big] \Big), \tag{2}$$

where ℓ_δ is the Huber loss and d indicates a genuine termination. Section ?? ablates the target network to show what it buys.

D. REINFORCE with a baseline

REINFORCE [4] optimises a parameterised policy π_θ directly, following the policy gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \, (G_t - b(s_t)) \right], \tag{3}$$

where G_t is the Monte-Carlo return from timestep t and $b(s_t)$ is a baseline. Any function of the state alone leaves the gradient unbiased, so the baseline is free to reduce variance; the standard choice, adopted here, is a learned state-value estimate $\hat{V}_\phi(s)$ trained by regression on the observed returns [5].

The baseline is not cosmetic. Because every CartPole reward is $+1$, the raw return G_t is large and strictly positive, so an unbaselined update increases the log-probability of every action taken in a successful episode, including the poor ones—the gradient carries the correct signal only through differences in magnitude. Subtracting $\hat{V}_\phi(s)$ centres the signal so that only better-than-expected actions are reinforced. This is ablated in Section ??.

Note that returns are deliberately not whitened. Dividing by the batch standard deviation, common in tutorial implementations, rescales the gradient as well as centring it; it is an adaptive step size rather than a baseline, and it would confound the ablation.

III. Experimental Design

The design follows the reproducibility recommendations of Henderson et al. [6] and the statistical recommendations of Agarwal et al. [7].

A. Shared budget and a single training loop

All three agents run under one training loop, differing only in what they do with a transition. Each run receives exactly 100k environment steps, and learning curves are plotted against environment steps rather than episodes. The distinction matters: a CartPole episode lasts around 20 steps for an untrained policy and 500 for a good one, so an episode budget silently grants the better method up to twenty-five times more data.

The loop bootstraps through a time-limit truncation but not through a genuine termination. Treating the 500-step truncation as terminal would teach both value-based methods that balancing the pole indefinitely is worth nothing.

B. Evaluation protocol

Training return is a poor measure of policy quality, because it includes the cost of exploration and therefore penalises whichever method explores most. Every 2k steps, training is paused and the agent is evaluated for 10 episodes with exploration disabled: greedy action selection for the two value-based methods, and the mode of the action distribution for REINFORCE.

The evaluation episodes use a fixed set of environment seeds, identical for every method, every seed and every evaluation point. The evaluation set is therefore held out and constant, so a difference between two curves reflects the policies rather than a difference in the states they happened to be tested on.

C. Equal-budget hyperparameter search

Each method receives 12 random-search trials drawn from a discrete space of its own hyperparameters, evaluated on 3 tuning seeds and scored by area under the evaluation curve—a criterion that rewards reaching a high return and reaching it early. The winning configuration is then frozen and run on 10 fresh evaluation seeds. Tuning seeds and evaluation seeds are disjoint, so no reported result is tuned on its own test set. The full search spaces and the selected values appear in Appendix A.

D. Statistics

Reinforcement learning results across seeds are skewed and frequently bimodal: a run either solves the task or collapses. The sample mean describes neither mode, and one failed seed in ten moves it substantially. Following Agarwal et al. [7], the point estimate is the interquartile mean (IQM), the mean of the middle 50% of the seeds, and uncertainty is reported as a 95% percentile bootstrap interval resampled over seeds. Per-seed final performance is also plotted directly, so that any bimodality remains visible rather than being absorbed into an aggregate.

IV. Implementation

The implementation is a single Python package. All three agents expose the same four-operation interface—select an action, accept a transition, finish an episode, report diagnostics—so the training loop, the evaluation procedure and the logging are written once and shared. Every behavioural difference between the methods lives in the update.

Both neural methods use a two-layer multilayer perceptron with 128 hidden units per layer and ReLU activations, optimised with Adam. DQN uses this network for Q_θ and a periodically synchronised copy for Q_{θ^-}; REINFORCE uses one for the policy logits and a second, separately optimised network for the baseline $\hat{V}_\phi$.

One configuration object fully determines one run, and a hash of it names that run's output files. This makes the run matrix resume-safe—an interrupted sweep restarts without repeating completed work—and it means every curve in this paper can be traced to the exact settings that produced it. Runs execute in a process pool, one run per worker, each pinned to a single thread so that the reported wall-clock times are not distorted by workers competing for cores.

Correctness is protected at the points where an error would be silent rather than loud. A wrong sign in the advantage or an off-by-one in the discounted return does not raise an exception; it produces a plausible learning curve and a wrong paper. The discounted-return computation, the state discretiser's clipping behaviour, the replay buffer's contents under overwrite, and the aggregation statistics are therefore checked against hand-computed values, and every method is checked to reproduce its curve exactly from a fixed seed.

Every figure and table in this paper is generated from the raw result files, and every number quoted in the text is a macro produced by the same stage, so the prose cannot drift out of step with the data.

V. Results

VI. Extended Analysis

VII. Discussion

VIII. Threats to Validity

A single environment. Every result here comes from CartPole-v1. The task is low-dimensional, densely rewarded, and solvable by a linear policy, and it is therefore the most favourable possible setting for the tabular method: four dimensions is few enough to discretise at all. On a task with a higher-dimensional observation, the table would grow exponentially while the two neural methods would be unchanged in size. The direction of that effect is predictable, but its magnitude is not measured here, and no result in this paper should be read as a claim about reinforcement learning in general.

Dense reward. CartPole gives a reward at every timestep, which suits Monte-Carlo policy gradients particularly well. Under a sparse reward, REINFORCE receives an informative gradient only from the rare successful episode, and the comparison could reverse. Testing that would need a second environment.

Search budget. The hyperparameter search is equal across methods by construction, but it is small—12 trials over a discrete grid. A larger budget could favour whichever method has the most delicate configuration, most plausibly DQN, which has twice as many hyperparameters as REINFORCE. The equality of the budget is defensible; its size is a limitation.

Ten seeds. 10 seeds is enough for an interquartile mean to be meaningful and enough to expose a bimodal outcome, but it is too few for well-powered significance testing between arms whose intervals overlap. No pairwise significance claim is made anywhere in this paper for that reason.

Implementation cost is partly subjective. Source lines and hyperparameter count are objective once the counting rule is fixed, but the rule involves judgement: the shared training loop is excluded, and a differently factored implementation would produce different numbers. The column is best read as an ordering, not as a measurement.

Wall-clock time is machine-specific. All runs executed on the same machine, one thread each, so the comparison between methods is fair; the absolute values are not portable, and the ratio would change on a GPU—where the two neural methods would gain and the tabular method would not.

IX. Conclusion

References

[1] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," IEEE Transactions on Systems, Man, and Cybernetics, vol. SMC-13, no. 5, pp. 834–846, 1983.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3–4, pp. 279–292, 1992.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, 2015.

[4] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," Machine Learning, vol. 8, no. 3–4, pp. 229–256, 1992.

[5] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

TABLE I
stub

A	B	C
x	y	z

[6] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, no. 1, 2018, pp. 3207–3214.
[7] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in Advances in Neural Information Processing Systems, vol. 34, 2021, pp. 29 304–29 320.

Appendix A
Hyperparameter Settings